Splay（伸展）树

1. 设计哲学：为什么需要Splay树?

1.1 计算机科学的"局部性原理"

Splay树的核心思想基于两个观察到的计算机程序行为规律：

1. **时间局部性**：刚刚被访问的数据，很可能在不久的将来再次被访问
2. **空间局部性**：一个数据被访问，其附近的数据也可能被访问

Splay树是一种平衡二叉查找树，最大的特点:通过 伸展（splay）操作把某个结点往上旋转到指定位置，特别是可以旋转到根的位置，成为新的根结点。使得整棵树仍然满足二叉查找树的性质。

常见的应用:如果需要经常查询和使用一个数，那么把它旋转到根结点，下次访问它，只需要查一次就找到了。

它的查找、插入、删除、分割、合并等操作，复杂度都是0(logn)的。并且保持平衡而不至于退化为链.

1.2 FHQ Treap树和Splay树的对比

FHQ和Splay的相同之处:

都很适合做分裂、合并。

- FHQ的基本操作是分裂和合并。
- Splay的基本操作是把某个结点通过旋转提升为根结点，即“提根”。

分裂:先把x旋转到根，然后分为两棵树L、R，L上结点的键值小于R上结点的键值。

合并L和R:把R的最小结点x旋转到根，此时x的左儿子为空，然后把L挂到x的左儿子上。

FHQ和Splay的不同:

- Splay和FHQ Treap的效率和维护BST的平衡性有关，都是O(logn)的。Treap给结点随机分配优先级来实现平衡性，在概率期望上是O(logn)的;Splay在旋转过程中隐含地达到了平衡，性能也是O(logn)的。
- 有些场合必须使用Splay，例如动态树LCT（link cut tree）。
- FHQ Treap能做可持久化，而Splay不适合做持久化。可持久化的关键是要用较小的空间来存储相邻时间点的两棵树的变化，避免空间爆炸。FHQ Treap是“无旋”的，相邻时间点的两棵树的形态差异较小。而Splay的基本操作是旋转，导致相邻时间点的两棵树差异很大。

2. Splay旋转

2.1 与普通BST旋转的本质区别

普通平衡树（如AVL、红黑树）的旋转是为了**维持平衡条件**，而Splay树的旋转是为了**优化未来访问**：

旋转目的	普通平衡树	Splay树
主要目标	保持高度平衡	将最近访问节点移到根

旋转目的	普通平衡树	Splay树
触发条件	插入/删除破坏平衡	任何访问操作

2.2 旋转

如何设计将一个节点x旋转到根的方法？ 需要达到以下两个目的。

- 每次旋转，节点x就上升一层，从而能在有限次操作后到达根部
- 旋转能改善BST的 平衡性，即尽量使二叉树层次变少，从根到叶子节点的路径变短，平均的层数和路径长度为O(logn)

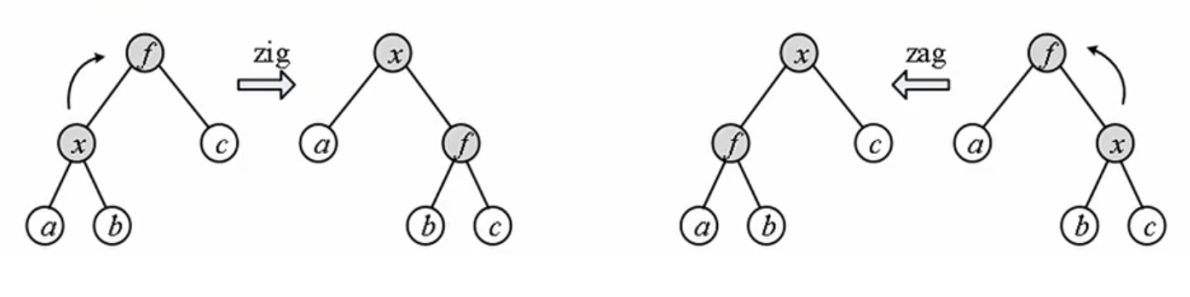
如果只考虑目的一，那么使用Treap树的旋转法即可，每次x与它的父节点交换位置，上升一层。成这种旋转为“单旋”，单旋不会减少二叉树的层数，对改善平衡性没有帮助。

Splay树主要使用“双旋”即伸展(Splay)，也就是两次单旋，同时旋转3个节点：x、父亲f、祖父g。双旋分为两种：一字旋、之字旋，能改善平衡性。

单旋

按旋转方向，把单旋分为左旋zag和右旋zig。

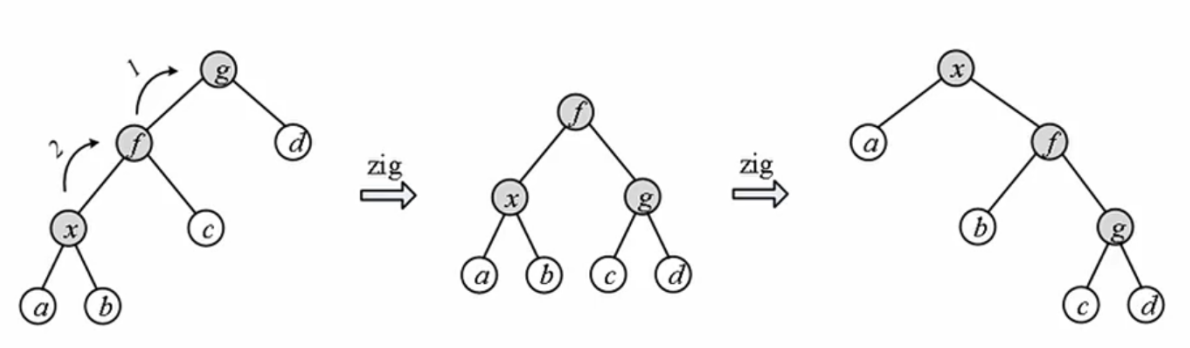
做一次zig或zag。此时x已经旋转上升到距离根只有一层的位置，父结点f就是根，只需要做一次单旋即可，和Treap树中的旋转法完全一样



一字旋

此时x、f、g在一条线上，做双旋zig-zig或zag-zag。

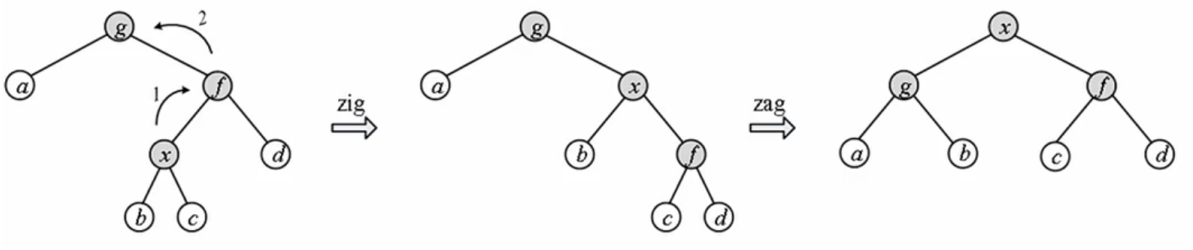
若x是f的左儿子，f是g的左儿子，做两次zig；若都是右儿子，做两次zag。



之字旋

此时x、f、g不在一条线上，做双旋zig-zag或zag-zig。

若x是f的左儿子，f是g的右儿子，先zig和做zag;否则先zag再zig。



3. 旋转的复杂度

旋转一次的时间是个常数。

把x从所在的深度提到根，总复杂度是多少?

如果是平衡二叉树，最深的结点深度是0(logn)，总复杂度就是0(logn)。

在均摊意义上，可以把Splay提根操作的复杂度看成是0(logn)。

详细内容可见《算法导论》（Thomas H.Cormen等著）第17章“平摊分析”，或者《算法竞赛》（罗勇军、郭卫斌著）第4.16.2节 “Splay树的平摊分析”。

4. Splay树的常用操作

Splay树常用于区间分裂和合并问题，旋转到根的功能使分裂和合并很容易实现。

例如，一个常见的区间操作，修改或查询区间[L,R]， 用Splay树很容易实现：先把节点L-1旋转到根；然后把节点R+1旋转到L-1的右子树上，此时R+1的左子树就是区间[L,R]。

4.1 旋转

splay(x，p)，把结点x旋转到位置p。p=0表示把x旋到根，x是整棵树的根;p!=0，表示把x旋到p的子树上，p成为x的父亲。

4.2 分裂

split(p，&L，&R)，p是排名或键值，按p把树分裂为两棵树L、R。例如p是排名，把第p个结点x旋到根，然后断开x和右子树，以x为根的是L，右子树是R。

4.3 合并

Merge(L，R)，合并两棵树L、R。缺省条件是L的所有结点比R小。首先把R的最小结点x旋转到根，此时x没有左儿子，然后让L做x的左儿子，就合并了两棵树。也可以把L的最大结点旋转到根，后续操作类似。

4.4 插入

Insert()，先把新结点按BST的规则插入到一个空的叶子结点上，然后旋转到根。

4.5 删除

del()，把要删除的结点旋到根，删除它，然后合并它的左右儿子。

5. 例题 P4008 文本编辑器

```cpp
#include<bits/stdc++.h>
using namespace std;
const int M = 2e6+10;
int cnt, root;
struct Node{int fa,ls,rs,size; char val;}t[M];   //tree[]存树;
void Update(int u){t[u].size = t[t[u].ls].size + t[t[u].rs].size+1;}  //用于排名
char str[M]={0};                //一次输入的字符串
int build(int L,int R,int f){   //把字符串str[]建成平衡树
    if(L>R) return 0;
    int mid = (L+R)>>1;
    int cur = ++cnt;
    t[cur].fa = f;
    t[cur].val= str[mid];
    t[cur].ls = build(L,mid-1,cur);
    t[cur].rs = build(mid+1,R,cur);
    Update(cur);
    return cur;                 //返回新树的根
}
int get(int x){return t[t[x].fa].rs == x;} //如果u是右儿子，返回1；左儿子返回0
void rotate(int x){                        //单旋一次
    int f=t[x].fa, g=t[f].fa, son=get(x);  //f:父亲；g：祖父
    if(son==1) {                           //x是右儿子，左旋zag
        t[f].rs = t[x].ls;                 //图“单旋”中的结点b
        if(t[f].rs)  t[t[f].rs].fa = f;
    }
    else {                                 //x是左儿子，右旋zig
        t[f].ls = t[x].rs;
        if(t[f].ls)  t[t[f].ls].fa = f;
    }
    t[f].fa=x;                  //x旋为f的父结点
    if(son == 1) t[x].ls=f;     //左旋，f变为x的左儿子
    else t[x].rs=f;             //右旋，f变为x的右儿子
    t[x].fa = g;                //x现在是祖父的儿子
    if(g){                      //更新祖父的儿子
        if(t[g].rs==f)  t[g].rs = x;
        else  t[g].ls = x;
    }
    Update(f);   Update(x);
}
void splay(int x,int goal){     //goal=0,新的根是x；goal!=0,把x旋为goal的儿子
    if(goal == 0) root=x;
    while(1){
        int f = t[x].fa, g=t[f].fa;        //一次处理x,f,g三个点
        if(f == goal) break;
        if(g != goal) {                    //有祖父，分一字旋和之字旋两种情况
            if(get(x)==get(f)) rotate(f);  //一字旋，先旋f、g
            else rotate(x);}               //之字旋，直接旋x
        rotate(x);
    }
    Update(x);
}
int kth(int k,int u){            //第k大数的位置
    if( k == t[t[u].ls].size + 1) return u;
    if( k <= t[t[u].ls].size )    return kth(k,t[u].ls);
    if( k >= t[t[u].ls].size + 1) return kth(k-t[t[u].ls].size-1,t[u].rs);
```

```
}
void Insert(int L,int len){                    //插入一段区间
    int x = kth(L,root), y = kth(L+1,root);  //x:第L个数的位置，y:第L+1个数的位置
    splay(x,0);   splay(y,x);                  //分裂
     //先把x旋到根，然后把y旋到x的儿子，此时y是x的右儿子，且y的左儿子为空
    t[y].ls = build(1,len,y);                  //合并：建一棵树，挂到y的左儿子上
    Update(y);    Update(x);
}
void del(int L,int R){              //删除区间[L+1,R]
    int x=kth(L,root), y=kth(R+1,root);
    splay(x,0); splay(y,x);         //y是x的右儿子，y的左儿子是待删除的区间
    t[y].ls=0;                  //剪断左子树，等于直接删除。这里为了简单，没有释放空间
    Update(y);    Update(x);
}
void inorder(int u){            //中序遍历
    if(u==0) return;
    inorder(t[u].ls);cout<<t[u].val;inorder(t[u].rs);
}
int main(){
    t[1].size=2;    t[1].ls=2;  //小技巧：虚拟祖父，防止旋转时越界而出错
    t[2].size=1;    t[2].fa=1;  //小技巧：虚拟父亲
    root=1; cnt=2;      //在操作过程中，root将指向字符串的根
    int pos=1;          //光标位置
    int n;   cin>>n;
    while(n--){
        int len;  char opt[10];  cin>>opt;
        if(opt[0]=='I'){
            cin>>len;
            for(int i=1;i<=len;i++){
                char ch=getchar();   while(ch<32||ch>126)  ch=getchar();
                str[i] = ch;
            }
            Insert(pos,len);
        }
        if(opt[0]=='D'){ cin>>len; del(pos,pos+len);}    //删除区间[pos+1,pos+len]
        if(opt[0]=='G'){
            cin>>len;     int x=kth(pos,root), y=kth(pos+len+1,root);
            splay(x,0);   splay(y,x);
            inorder(t[y].ls);    cout<<"\n";
        }
        if(opt[0]=='M'){ cin>>len; pos=len+1;}
        if(opt[0]=='P') pos--;
        if(opt[0]=='N') pos++;
    }
}
```